AI-BASED NETWORK INTRUSION DETECTION & TRAFFIC MONITORING

Complete implementation report · 20 planned steps plus the dark UI and simulation extension

Executive summary

This project is an authenticated network-traffic analysis application. It replays authorized PCAP/PCAPNG data, parses packet metadata without retaining payloads, groups packets into bidirectional flows, computes versioned features, applies explainable rules, optionally runs a verified machine-learning model, persists results to PostgreSQL, and presents stored evidence in a dashboard.

Evidence boundary: no real labeled dataset or trained project model was supplied. Synthetic fixtures validate software wiring only; they do not establish detection accuracy or real attack identification.

PACKET CAPTURE	METADATA PARSE	FLOW BUILDER	FEATURES + MODEL	RULE ALERTS	POSTGRES + API	DASHBOARD
PCAP / PCAPNG	payload omitted	bidirectional	schema 1.0	evidence attached	authenticated	persisted views

Project status

The twenty-step plan is implemented and pushed to the user-provided GitHub repository. This additional update adds a graphite/brass dashboard theme and an authenticated synthetic simulation route. The local Compose services were rebuilt and reported healthy.

The dashboard does not capture traffic on its own. Monitoring requires an operator to replay an authorized capture or configure an explicitly authorized live-capture environment.

System scope and current state

Core capabilities

Packet processing	Metadata-only IPv4/IPv6 parsing and streaming offline PCAP/PCAPNG replay.
Flow and features	Bounded bidirectional aggregation, directional counters, event-time expiry and feature schema 1.0.
Detection	Trusted-model verification with explicit unavailable states; rule-only SYN-burst and high packet-rate alerts.
Persistence and API	PostgreSQL migrations, transactional observation persistence, authenticated REST and alert WebSockets.
Dashboard	Overview, capture history, model/evaluation, alerts, flows, user administration and the new simulation route.
Operations	Compose services, health checks, reproducible synthetic capture and isolated benchmark workflow.

Local runtime snapshot

Component	State / note
Compose project	nids-local; PostgreSQL, API and dashboard reported healthy after rebuild.
Dashboard	http://localhost:8080 · authenticated route.
Persistent database	Existing volume preserved; no new capture was replayed into it.
Synthetic simulation	Request-local only; no database write.
Account credentials	Kept out of this report and source control.

Implementation steps - 1-10

Each planned step was checkpointed in Git. See docs/PROGRESS.md for commit IDs and the detailed verification record.

01 · Audit and foundation

Requirements, environment audit, safe Git defaults and initial checkpoint.

02 · Packet ingestion and replay

Metadata parsing, PCAP/PCAPNG streaming, filtering and explicit optional capture.

03 · Flows and feature schema

Bounded bidirectional flow builder, directional counters and schema 1.0.

04 · Dataset and evaluation

Feature export, provenance, validation, capture-group splits and training workflow.

05 · Inference and alerts

Verified-model checks, rule-only behavior, explainable thresholds and deduplication.

06 · Persistence and REST

PostgreSQL migrations, transactional ingestion and validated authenticated API.

07 · WebSockets and access

Roles, token sessions, audit events, acknowledgement and alert streaming.

08 · Authenticated dashboard

React alert/flow screens, URL filters, responsive states and session controls.

09 · Reproducible deployment

Docker Compose services, health checks, optional monitor and operator guides.

10 · Integration and handoff

Initial end-to-end verification, evidence limits and first release handoff.

Implementation steps - 11-20

Each planned step was checkpointed in Git. See docs/PROGRESS.md for commit IDs and the detailed verification record.

11 · PostgreSQL integration

Opt-in PostgreSQL coverage for authentication, persistence and WebSocket behavior.

12 · Capture lifecycle

Run state, counters, model identity, timing and bounded failure codes.

13 · Retry-safe replay

Stable capture identity, idempotency, duplicate counts and conflict rejection.

14 · Capture history API

Authenticated paginated list/detail APIs with status and time filtering.

15 · Analytics API

Persisted summary plus hourly/daily UTC timelines and range validation.

16 · Overview and history

Persisted dashboard analytics, accessible chart data and capture lifecycle details.

17 · Model/evaluation view

Verified model/report status and metrics only when report identity/checksum match.

18 · User administration

Admin-only account management, last-admin protection, audit and session revocation.

19 · Login throttling

Bounded process-local abuse control, 429 responses and proxy boundary documentation.

20 · Performance and handoff

Synthetic generator/benchmark, lazy-loaded frontend chunks and final evidence handoff.

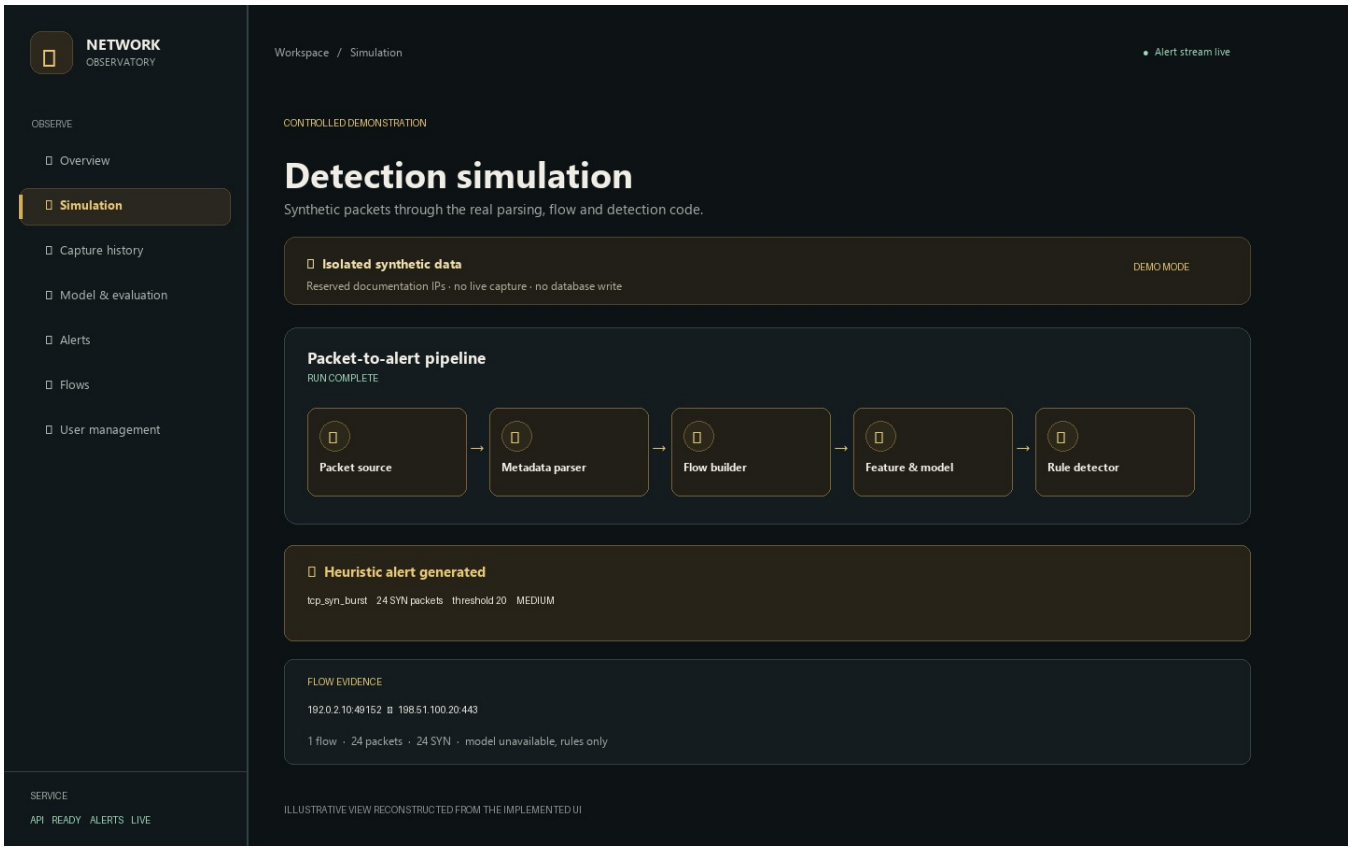
Simulation visualization and theme update

The visual system now uses a graphite canvas, raised charcoal surfaces, warm white text, brass actions, and separate severity colors. The design and UX contracts record the palette, route, reduced-motion behavior, and evidence boundary.

End-to-end simulation

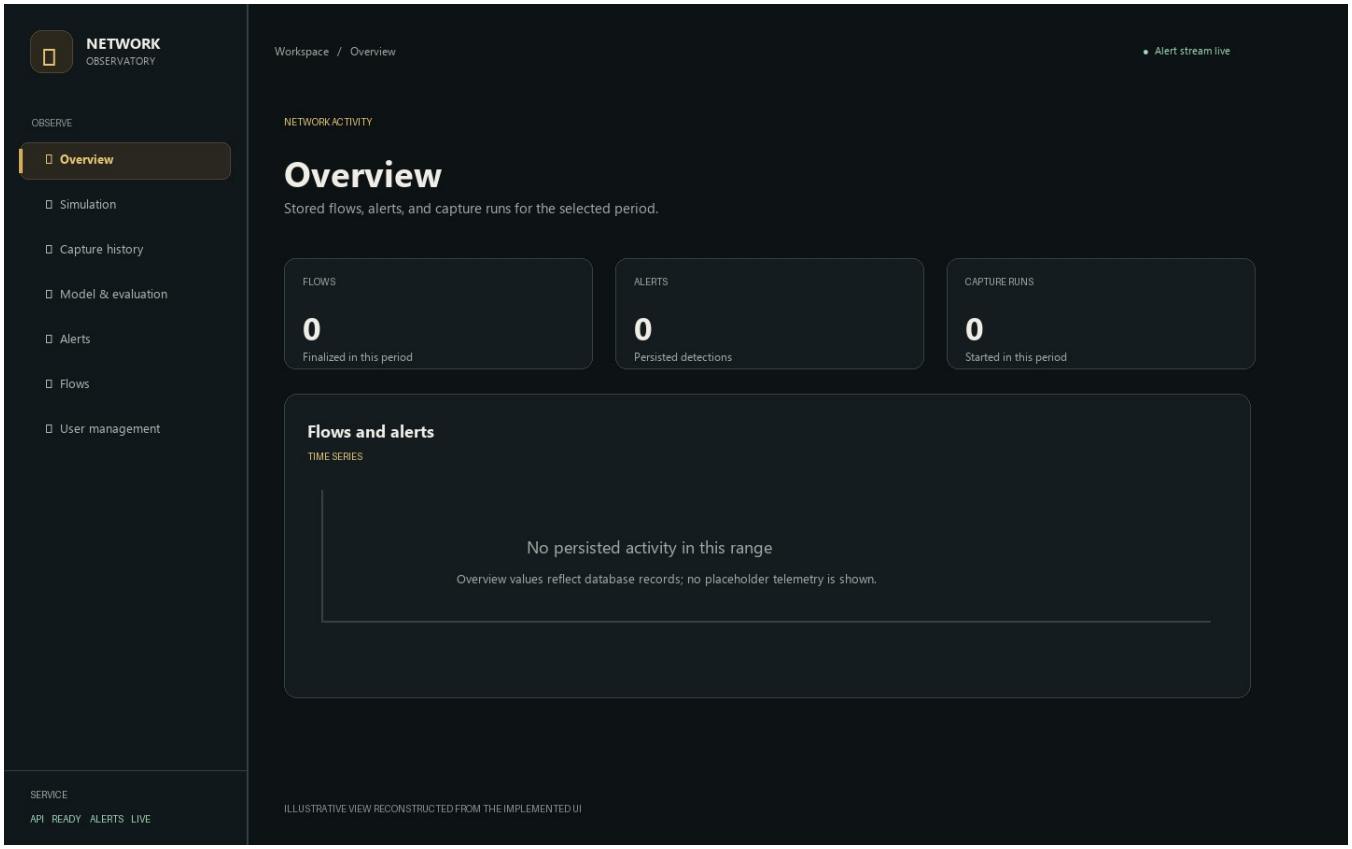
An authenticated POST /api/v1/simulation request generates 24 bounded synthetic TCP SYN packets at documentation-only addresses. It uses the real packet metadata parser, FlowBuilder, InferenceEngine availability state, and AlertEngine. The run creates one bidirectional flow and a medium tcp_syn_burst alert because the 24 SYN packets exceed the configured threshold of 20.

The API does not store the simulation in PostgreSQL. The Model status is unavailable because no trusted model is configured, so the shown event is a rule-based heuristic review signal. It does not confirm a real attack.



Illustrative reconstruction of the implemented simulation screen. The completed page was also run and viewed in the local app during this handoff. All IPs shown are documentation addresses.

Dashboard view and operating notes



Illustrative reconstruction of the Overview screen. Its displayed values come from persisted records; zero counts indicate an empty selected period.

How to view the application

Open http://localhost:8080 and sign in with the local administrator account already provisioned for this workspace. Choose **Simulation** and press **Run simulation** to see the deterministic demo. The account password is intentionally excluded from the repository and report.

For authorized capture replay, use docs/DEPLOYMENT.md. Keep the existing nids-local database volume: the simulation does not seed it, and no volume-removal command was used.

Verification and results

Recorded project verification

Area	Recorded result	Interpretation
Backend suite	74 passed, 2 skipped	Step 20 full suite; PostgreSQL integrations ran separately.
PostgreSQL integration	2 passed in Step 18	Disposable test environment; cleaned after checks.
Frontend suite	15 passed in 3 files	Step 20 frontend regression coverage.
Frontend build	TypeScript/build passed	Current build: 268.45 kB main JS; 337.60 kB chart chunk.
Compose	Images built; services healthy	PostgreSQL, API and dashboard healthy after this update.
Simulation API	24 packets → 1 flow → tcp_syn_burst	24 SYN at threshold 20; persisted=false.

Synthetic benchmark

The isolated Step 20 benchmark used five runs of 2,400 generated packets each. Median end-to-end time was 23.765572 seconds, or 100.99 packets/second on the recorded CPython 3.12.15 Linux/WSL2 environment. Startup and output time are included. This is not live capture capacity, tail latency, model throughput, or attack-detection evidence.

Current UI update verification

The frontend type check and production build passed. Docker Compose rebuild succeeded and PostgreSQL, API and dashboard health checks passed. Direct execution of the simulation pipeline returned 24 packets, one flow, one rule alert and persisted=false.

Research boundaries and next evidence

What is not established

No provenance-tracked, labeled real dataset or trained trusted project model is present. Therefore the report makes no real-data precision, recall, F1, classifier accuracy, false-positive rate, or generalization claim. Live capture has not been verified on this host. Public deployment still needs TLS and a security review; login throttling is process-local.

- A legally usable, provenance-tracked dataset whose inputs match this project’s feature schema.
- Independent capture groups for a held-out test and reproducible evaluation.
- A trained model plus verified metadata and an integrity-matched evaluation report.
- A supported capture driver and explicitly authorized interface to evaluate live capture.
- TLS, deployment-specific review, and distributed abuse controls before public deployment.

Git and project references

Repository: github.com/sugarfreelife/-AI-Based-Real-Time-Network-Intrusion-Detection-Traffic-Monitoring-System · branch main. The previously completed Step 20 release was commit 388ae15. Current theme, simulation, report, and presentation are delivered as subsequent checkpoints.

Project context: PROJECT_CONTEXT.md · implementation log: docs/PROGRESS.md · test record: docs/TESTING.md · handoff: docs/HANDOFF.md · design contract: DESIGN.md · UX contract: UX-CONTRACT.md.

Handling note

Use generated synthetic inputs only for demonstrations. The simulation illustration is explicitly labeled, and its heuristic event must not be represented as a true intrusion. Keep local authentication secrets outside tracked documentation.